

Reviewer Pack — Submodular Matroid Rank Defect for Monotone CLIQUE

Entry 8c018be050c5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 05:58:03 UTC

Submodular Matroid Rank Defect for Monotone CLIQUE

Entry ID: 8c018be050c5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 05:58:03 UTC

1. Conjecture

Field A (mathematical branch): matroid theory **Field B** (complexity object): monotone circuit complexity

Statement:

There exists a submodular function μ on monotone DNF formulas such that $\mu(F) \leq O(\log n)$ for any DNF of size $\text{poly}(n)$, $\mu(k\text{-CLIQUE}) = \Omega(n)$, and $\mu(F \wedge G) \leq \mu(F) + \mu(G) - 1$ for all DNFs F, G .

Rationale (proposer’s reasoning):

This conjecture leverages matroid rank submodularity to quantify the structural complexity of monotone DNFs. The submodularity ensures that redundant terms in the DNF do not disproportionately inflate the measure, while the CLIQUE’s exponential term dependency forces μ to scale with n . This aligns with Razborov’s polymatroid expansion framework but avoids algebraic geometry barriers.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8170f234ebee5fab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, k):
        terms = []
        for _ in range(k):
            term = [random.choice([0, 1]) for _ in range(n)]
            terms.append(term)
        return terms

    def dnf_to_matroid_rank(dnf):
        rank = len(set(tuple(term) for term in dnf))
        return rank

    def submodular_function(dnf):
        return min(dnf_to_matroid_rank(dnf), math.log(len(dnf)))

    def is_submodular(F, G):
        return submodular_function(F) + submodular_function(G) - 1 >= submodular_function(F + G)

    def test_submodularity():
        for _ in range(100):
            F = generate_dnf(n, random.randint(5, 20))
            G = generate_dnf(n, random.randint(5, 20))
            if not is_submodular(F, G):
                return False
        return True

    def test_clique():
        for n in range(10, 41):
            dnf = [[1] * n]
            mu = submodular_function(dnf)
            if mu != n:
                return False
        return True

    n = random.randint(5, 40)
```

```

F = generate_dnf(n, random.randint(5, 20))
mu_F = submodular_function(F)

if not test_submodularity():
    return {
        "metric_name": "submodularity_test",
        "metric_value": None,
        "instances_tested": 100,
        "conjecture_holds": False,
        "counterexample": "Submodularity test failed"
    }

if not test_clique():
    return {
        "metric_name": "clique_test",
        "metric_value": None,
        "instances_tested": 40,
        "conjecture_holds": False,
        "counterexample": "Clique test failed"
    }

return {
    "metric_name": "submodular_function_value",
    "metric_value": mu_F,
    "instances_tested": 1,
    "conjecture_holds": mu_F <= math.log(n),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(r["conjecture_holds"] for r in results if r["conjecture_holds"]) / len

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["coun
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
erexample': 'Submodularity test failed'}
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold
TRIAL: {'metric_name': 'submodularity_test', 'metric_value': None, 'instances_tested': 100, 'conject
TRIAL: {'metric_name': 'clique_test', 'metric_value': None, 'instances_tested': 40, 'conjecture_hold

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ed7ced5.py", line 103, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ed7ced5.py", line 103, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before collecting data; cannot verify support fraction or counterexamples |
next: Fix the test to handle seed generation properly and re-run with sufficient iterations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42219
2	preregistration	ollama_remote	qwen3:8b	0	0	27727
3	novelty	ollama_remote	qwen3:8b	0	0	22574
4	novelty	ollama_remote	qwen3:8b	0	0	16300
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44175
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9955
7	judge	ollama_remote	qwen3:8b	0	0	14355

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 177304 ms total latency. Provider mix:
{'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/8c018be050c5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8c018be050c5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8c018be050c5.tar.gz` (if generated)